1)

```
C:\Users\BALM>arp -a

Interface: 192.168.29.118 --- 0xb
  Internet Address      Physical Address      Type
  192.168.29.1          fc-7a-58-5e-88-69     dynamic
  192.168.29.123        5c-02-14-3b-64-91     dynamic
  192.168.29.255        ff-ff-ff-ff-ff-ff     static
  224.0.0.22            01-00-5e-00-00-16     static
  224.0.0.251           01-00-5e-00-00-fb     static
  224.0.0.252           01-00-5e-00-00-fc     static
  239.255.255.250       01-00-5e-7f-ff-fa     static
  255.255.255.255       ff-ff-ff-ff-ff-ff     static

C:\Users\BALM>
```

2)

```
C:\Users\BALM>hostname
DESKTOP-S2U7JB9

C:\Users\BALM>
```

3)

```
C:\Users\BALM>ipconfig

Windows IP Configuration


Ethernet adapter Ethernet:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :

Wireless LAN adapter Local Area Connection* 9:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :

Wireless LAN adapter Local Area Connection* 10:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :

Wireless LAN adapter Wi-Fi:

   Connection-specific DNS Suffix  . :
   IPv6 Address. . . . . . . . . . . : 2405:201:e044:88d8:ef0c:2654:77d3:421
   Temporary IPv6 Address. . . . . . : 2405:201:e044:88d8:d515:e253:e938:2ddd
   Link-local IPv6 Address . . . . . : fe80::cf25:64c0:cf22:adda%11
   IPv4 Address. . . . . . . . . . . : 192.168.29.118
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . : fe80::fe7a:58ff:fe5e:8869%11
                                       192.168.29.1

Ethernet adapter Bluetooth Network Connection:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :

C:\Users\BALM>
```

4)

```
C:\Users\BALM>nbtstat -a

Displays protocol statistics and current TCP/IP connections using NBT
(NetBIOS over TCP/IP).

NBTSTAT [ [-a RemoteName] [-A IP address] [-c] [-n]
        [-r] [-R] [-RR] [-s] [-S] [interval] ]

  -a   (adapter status) Lists the remote machine's name table given its name
  -A   (Adapter status) Lists the remote machine's name table given its
                        IP address.
  -c   (cache)          Lists NBT's cache of remote [machine] names and their IP addresses
  -n   (names)          Lists local NetBIOS names.
  -r   (resolved)       Lists names resolved by broadcast and via WINS
  -R   (Reload)         Purges and reloads the remote cache name table
  -S   (Sessions)       Lists sessions table with the destination IP addresses
  -s   (sessions)       Lists sessions table converting destination IP
                        addresses to computer NETBIOS names.
  -RR  (ReleaseRefresh) Sends Name Release packets to WINS and then, starts Refresh

  RemoteName   Remote host machine name.
  IP address   Dotted decimal representation of the IP address.
  interval     Redisplays selected statistics, pausing interval seconds
               between each display. Press Ctrl+C to stop redisplaying
               statistics.
```

EX.NO 1

24/03/26 NETWORK COMMANDS USED IN LINUX

AIM: TO study various network connection used in Linux & windows system.

COMMANDS:

1) arp -a → Shows IP address of the computer along with the IP and MAC address
2) hostname → Displays name of the computer.
3) ipconfig → Displays configuration in the PC.
4) nbtstat -a → Helps to solve problems with NetBIOS over TCP IP.
5) nslookup → Displays informations of a DNS for a domain or IP address.
6) pathping → Traces the route to a destination and performs a test at each router along the path to identify packet loss and latency.
7) Ping → Tests connectivity between two devices on a network.

eg: ping hostname
ping
ping IP_address

5)

```
C:\Users\BALM>nslookup
Default Server:  reliance.reliance
Address:  2405:201:e044:88d8::c0a8:1d01
```

6)

```
C:\Users\BALM>pathping

Usage: pathping [-g host-list] [-h maximum_hops] [-i address] [-n]
                [-p period] [-q num_queries] [-w timeout]
                [-4] [-6] target_name

Options:
    -g host-list        Loose source route along host-list.
    -h maximum_hops     Maximum number of hops to search for target.
    -i address          Use the specified source address.
    -n                  Do not resolve addresses to hostnames.
    -p period           Wait period milliseconds between pings.
    -q num_queries      Number of queries per hop.
    -w timeout          Wait timeout milliseconds for each reply.
    -4                  Force using IPv4.
    -6                  Force using IPv6.
```

7)

```
C:\Users\BALM>ping

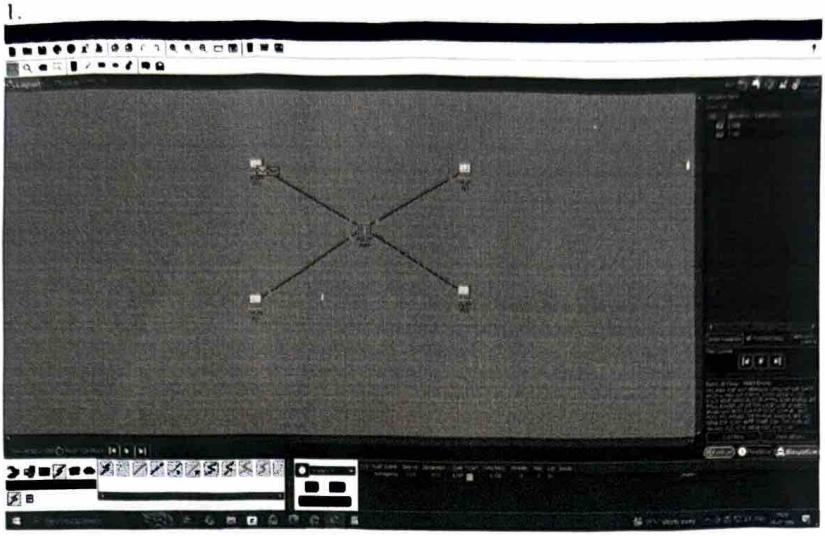
Usage: ping [-t] [-a] [-n count] [-l size] [-f] [-i TTL] [-v TOS]
            [-r count] [-s count] [[-j host-list] | [-k host-list]]
            [-w timeout] [-R] [-S srcaddr] [-c compartment] [-p]
            [-4] [-6] target_name

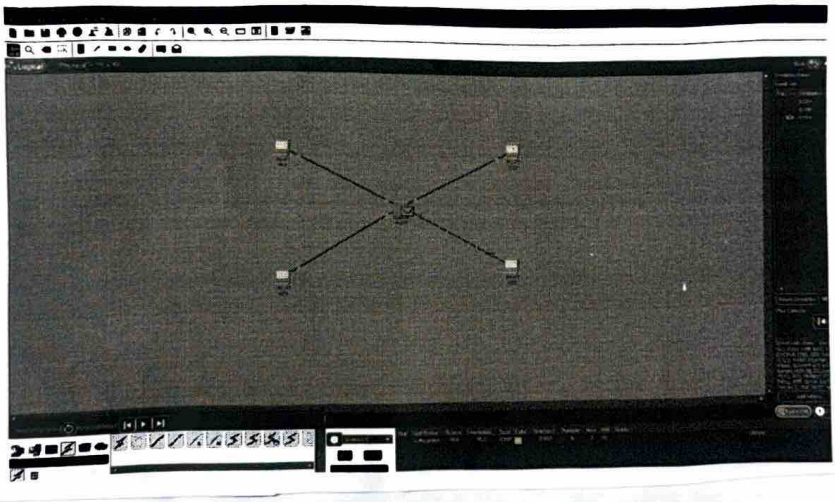
Options:
    -t              Ping the specified host until stopped.
                    To see statistics and continue - type Control-Break;
                    To stop - type Control-C.
    -a              Resolve addresses to hostnames.
    -n count        Number of echo requests to send.
    -l size         Send buffer size.
    -f              Set Don't Fragment flag in packet (IPv4-only).
    -i TTL          Time To Live.
    -v TOS          Type Of Service (IPv4-only. This setting has been deprecated
                    and has no effect on the type of service field in the IP
                    Header).
    -r count        Record route for count hops (IPv4-only).
    -s count        Timestamp for count hops (IPv4-only).
    -j host-list    Loose source route along host-list (IPv4-only).
    -k host-list    Strict source route along host-list (IPv4-only).
    -w timeout      Timeout in milliseconds to wait for each reply.
    -R              Use routing header to test reverse route also (IPv6-only).
                    Per RFC 5095 the use of this routing header has been
                    deprecated. Some systems may drop echo requests if
                    this header is used.
    -S srcaddr      Source address to use.
    -c compartment  Routing compartment identifier.
    -p              Ping a Hyper-V Network Virtualization provider address.
    -4              Force using IPv4.
    -6              Force using IPv6.
```

RESULT : Thus the network commands has been studied and executed successfully

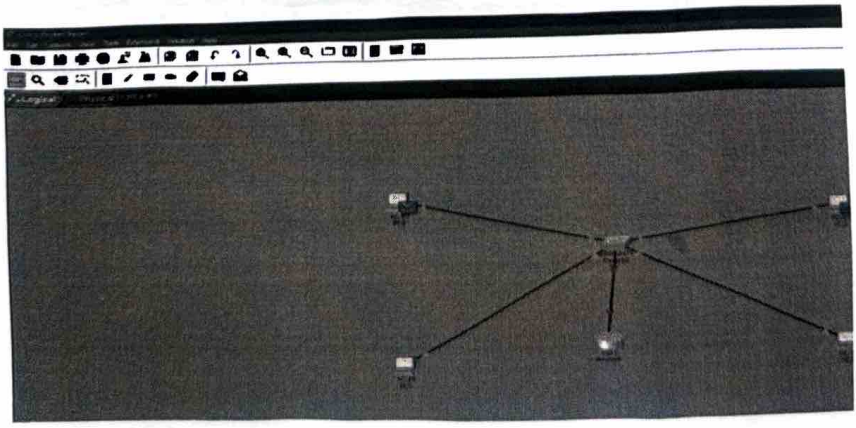
a) HUB TOPOLOGY :

1.





b) SWITCH AND ROUTER TOPOLOGY



EX NO.2

25/07/26 NETWORK TOPOLOGY USING HUB, SWITCH AND ROUTER

AIM: TO create and simulate simple network topologies using a Hub, switch & Router in Cisco Packet Tracer and study data communication between devices.

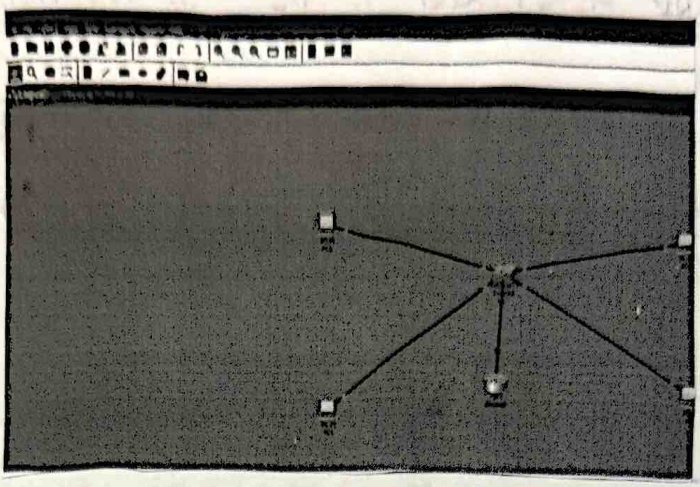
PROCEDURE:

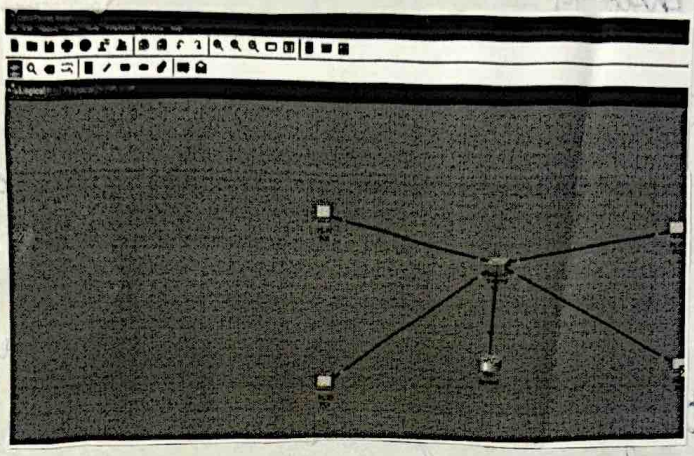
a) Hub Topology

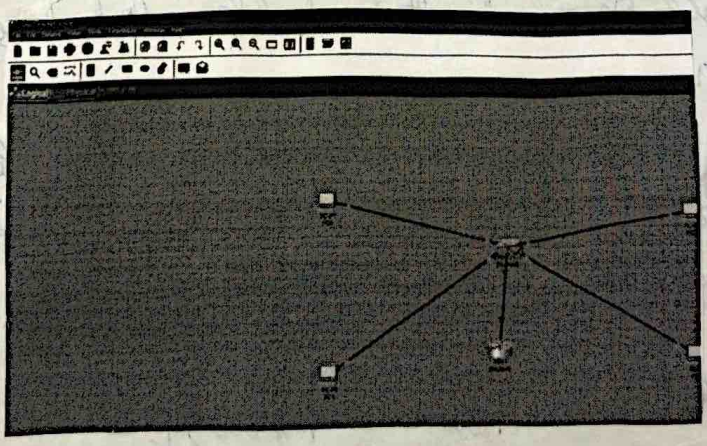
1. Opened Cisco Packet Tracer.
2. Placed 1 Hub & 4 PC's in the workspace.
3. Assigned IP addresses to all the 4 PC's
4. Connected all the PC's to the Hub using copper straight-Through cables.
5. Selected simple PDU (message icon) and clicked on the source PC and then double clicked on the destination PC.
6. selected the simulation mode and observed the Packet transmission and communication between the PC's.

b) Switch and Router Topology

1. Opened Cisco Packet Tracer.
2. Placed one Router, one switch and four PC's in the workspace.
3. Assigned the IP address to all the 4 PC's.
4. Connected all the 4 PC's to the switch and connected the switch to the Router using copper straight







through cables.

5. Selected the simple PDU and clicked on the source PC and then double clicked on the destination PC.

6. Selected the simulation mode and observed the packet transmission through the switch and Router

RESULT: The network topologies using Hubs, switch and Router were successfully created in Cisco Packet tracer and the PDU was transmitted successfully between the soure and destination devices.

EX.NO. 3 STUDY OF DIFFERENT ~~TYPES~~ TYPES OF NETWORK CABLES
1/08/2026

AIM: To study different types of Network cables such as UTP, STP, coaxial cable and Fibre optic cable.

a) Understand different types of network cable.

1. UTP : Unshielded Twisted Pair.

- categories : cat 3, Cat 5, Cat 5e
- Maximum speed : Cat 3 - 10 Mbps, cat 5 - 100 Mbps. cat 5e - 1 Gbps.
- Advantages : Low cost, Easy to install and maintain.
- Disadvantage : Prone to EMI, More affected by noise.
- Application : 10 Base -T Ethernet, Fast Ethernet.

2. STP : (Shielded Twisted Pair)

- category : cat 6, Cat 6a
- speed : Up to 10 Gbps
- Advantages : shielded from interference, Faster than UTP.
- Disadvantages : Expensive, Hard to install.
- Applications : Gigabit Ethernet, 10 G B ethernet, Data Centers.

3. SSTP (Screen shielded Twisted Pair)

- Category : Cat 7
- Speed : Upto 10 Gbps
- Advantages : Better shielding, Less interference.
- Disadvantages : Costly, Difficult to install.
- Applications : Gigabit Ethernet, 10 G Ethernet.

4. Coaxial Cable :

- Types : RG-6, RG-59, RG-11
- speed : 10 - 100 Mbps.
- Advantages : High bandwidth, Less interference.
- Disadvantages : Bulky, Limited Distance.
- Applications : Cable TV, Television Networks.

5. Fibre optics Cable :

- Types : Single mode, Multi mode.
- Speed : 100 Gbps.
- Advantages : High speed, High bandwidth etc.
- Disadvantages : Expensive, Required skilled installers.

(i) Make your own Ethernet Cross-over Cable/ straight cable.

Tools required : i) CAT 5/ cat5e cable,
ii) crimping tool
iii) 2 RJ45 connectors
iv) RJ45 boots/shield (optional)

Difference:

- Straight Cable: Same wiring (A-A) on both end
- Crossover Cable: one end A, other end B (A-B).

Steps:

1. Insert the cable through the RJ45 shield.
2. Strip about 1.5cm of the outer cable jacket.
3. Untwist the wires and arrange them:
   - Straight: A-A
   - Crossover: A-B.
4. Trim the wires evenly and insert them into the RJ45 connector.
5. Ensure the cable jacket reaches inside the connectors.
6. Crimp the connector using the crimping tool.
7. Repeat for the other end and test the cable by connecting two devices.

RESULT: The construction of Ethernet straight-through and crossover cables, their wiring standards, required tools, and preparation steps were studied.

EX.NO.4
1/08/26

CONFIGURATION OF LAN

AIM: Setup and configure a LAN (local area network) using a switch and Ethernet cables in your Lab.

LAN: A Local Area Network (LAN) is a network that connects computers and other devices within a limited geographical area such as a home, school or laboratory. It allows devices to communicate with each other and share resources.

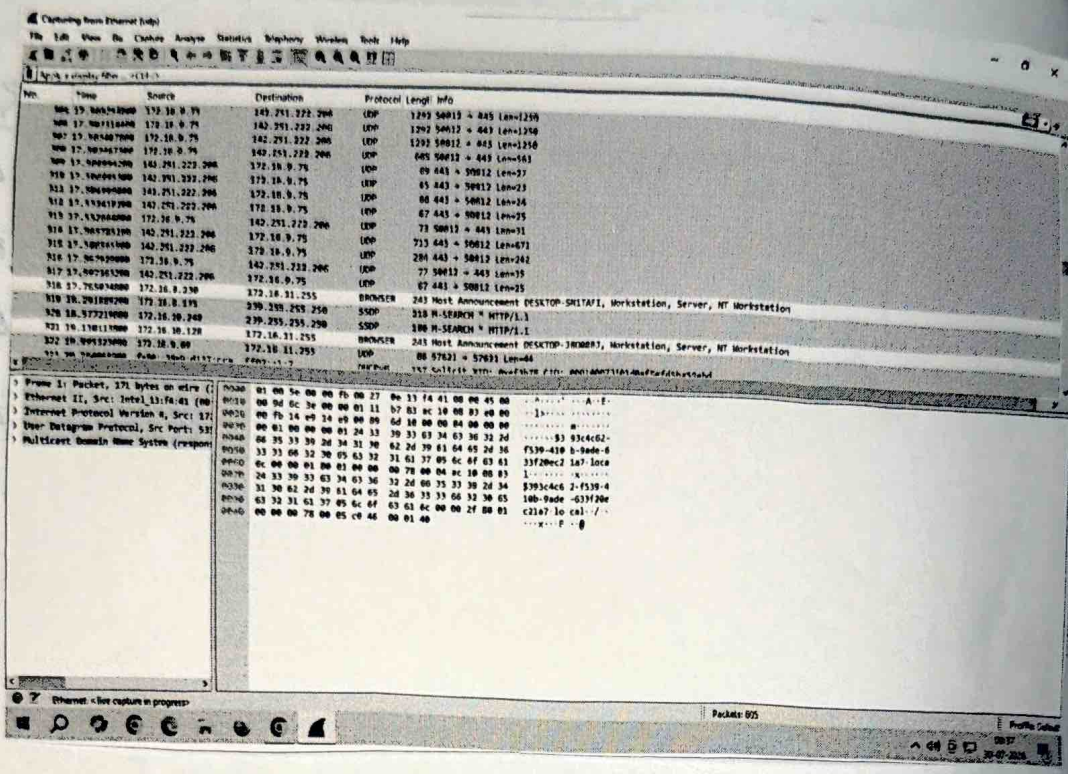
PROCEDURE:

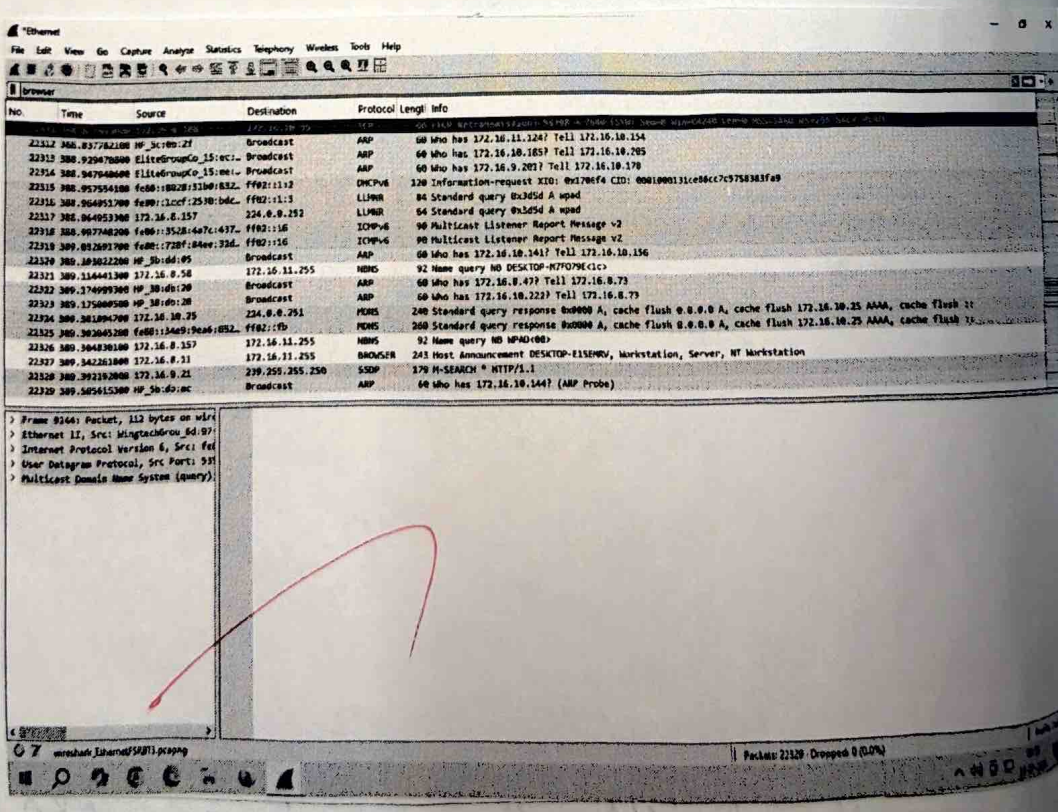
1. Connected 4 computers to a switch using Ethernet cables.
2. Assigned IP addresses:
   - PC1 - 10.1.1.1
   - PC2 - 10.1.1.2
   - PC3 - 10.1.1.3
   - PC4 - 10.1.1.4
   - Subnet Mask - 255.0.0.0
3. Configured the switch with IP addresses 10.1.1.5
4. Verified connectivity using the ping command.
5. Shared a folder and accessed it from the other PCs on the LAN.

IP Address Table :

Device	IP Address	Subnet Mask
PC1	10 . 1.1.1	255 . 0.0.0
PC2	10.1.1.2	255.0.0.0
PC 3	10.1.1.3	255.0.0.0
PC 4	10.1.1.4	255.0.0.0
switch	10.1.1.5	255.0.0.0

RESULT: A Local Area Network (LAN) was successfully set up and configured using a switch and Ethernet cable. Connectivity between all devices was verified using the ping command and file sharing is tested successfully.


Capturing from Ethernet [wlp]
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
No. Time Source Destination Protocol Length Info
Frame 1: Packet, 171 bytes on wire (
Ethernet II, Src: Intel_13:fa:d1 (98
Internet Protocol Version 4, Src: 17
User Datagram Protocol, Src Port: 53
Multicast Domain Name System (respon
Ethernet: <live capture in progress>
Packets: 895
Profile: Default


*Ethernet
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
browser
No. Time Source Destination Protocol Length Info
22312 388.837782100 HP_5c:00:2f Broadcast ARP 60 Who has 172.16.11.124? Tell 172.16.10.154
22313 388.929470600 EliteGroupCo_15:ec:.. Broadcast ARP 60 Who has 172.16.10.185? Tell 172.16.10.205
22314 388.947940600 EliteGroupCo_15:ec:.. Broadcast ARP 60 Who has 172.16.9.201? Tell 172.16.10.178
22315 388.957554100 fe80::8028:31b0:832.. ff02::1:3 DHCPv6 120 Information-request XID: 0x1706f4 CID: 000100013...
22316 388.964951700 fe80::1ccf:2530:bdc.. ff02::1:3 LLMNR 84 Standard query 0x3d5d A wpad
22317 388.964953300 172.16.8.157 224.0.0.252 LLMNR 64 Standard query 0x3d5d A wpad
22318 388.997748200 fe80::3528:4e7c:437.. ff02::16 ICMPv6 90 Multicast Listener Report Message v2
22319 389.052691700 fe80::728f:84ee:32d.. ff02::16 ICMPv6 90 Multicast Listener Report Message v2
22320 389.103022200 HP_5b:dd:05 Broadcast ARP 60 Who has 172.16.10.141? Tell 172.16.10.156
22321 389.114441300 172.16.8.58 172.16.11.255 NBNS 92 Name query NB DESKTOP-M7FO79E<1c>
22322 389.174999300 HP_38:db:20 Broadcast ARP 60 Who has 172.16.8.47? Tell 172.16.8.73
22323 389.175000500 HP_38:db:20 Broadcast ARP 60 Who has 172.16.10.222? Tell 172.16.8.73
22324 389.301894700 172.16.10.25 224.0.0.251 MDNS 240 Standard query response 0x0000 A, cache flush 0.0.0.0 A, cache flush 172.16.10.25 AAAA, cache flush 1...
22325 389.303045200 fe80::34e9:9ea6:852.. ff02::fb MDNS 260 Standard query response 0x0000 A, cache flush 0.0.0.0 A, cache flush 172.16.10.25 AAAA, cache flush 1...
22326 389.304830100 172.16.8.157 172.16.11.255 NBNS 92 Name query NB WPAD<00>
22327 389.342261000 172.16.8.31 172.16.11.255 BROWSER 243 Host Announcement DESKTOP-E1SEMRV, Workstation, Server, NT Workstation
22328 389.392192000 172.16.9.21 239.255.255.250 SSDP 179 M-SEARCH * HTTP/1.1
22329 389.505615300 HP_5b:d9:ec Broadcast ARP 60 Who has 172.16.10.144? (ARP Probe)
Frame 8144: Packet, 112 bytes on wire
Ethernet II, Src: WingtechGrou_6d:97:
Internet Protocol Version 6, Src: fe
User Datagram Protocol, Src Port: 53
Multicast Domain Name System (query)
wireshark_EthernetFSKBT1.pcapng
Packets: 22329 · Dropped: 0 (0.0%)

Ex.No.5
6/8/26

PACKET CAPTURE USING WIRESHARK

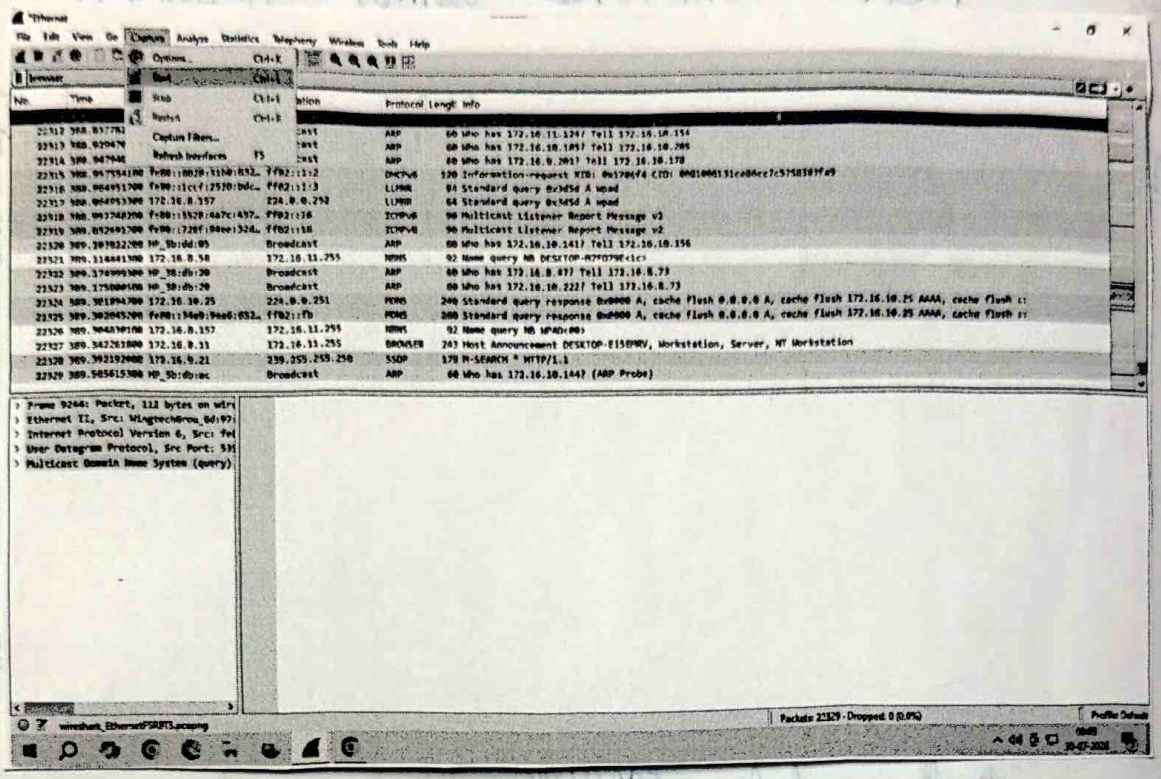
AIM: To capture, inspect and analyze network packets using the Wireshark packet analyzer.

ALGORITHM:

1. Open wireshark.
2. Select the active network interface.
3. Configure capture to stop after 100 packets.
4. Start packet capture.
5. Apply display filters.
6. Inspect packet details and view the flow graph.
7. Save the captured packets.

PROCEDURE:

1. Launch Wireshark and select the active network interface.
2. Go to Capture → options and set stop capture after 100 packets.
3. Click start to begin packet capture.
4. Apply filters:
   - TCP/UDP → tcp || udp
   - ARP → arp
   - DNS → dns


*Ethernet
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
Options... Ctrl+E
Start Ctrl+E
Stop Ctrl+E
Restart Ctrl+R
Capture Filters...
Refresh Interfaces F5
No. Time Protocol Length Info
ARP 60 Who has 172.16.11.124? Tell 172.16.10.154
ARP 60 Who has 172.16.10.185? Tell 172.16.10.205
ARP 60 Who has 172.16.9.201? Tell 172.16.10.178
DHCPv6 120 Information-request XID: 0x1704f4 CID: 00010001315ce084ec7c5758369ffa9
LLMNR 84 Standard query 0x345d A wpad
LLMNR 64 Standard query 0x345d A wpad
ICMPv6 90 Multicast Listener Report Message v2
ICMPv6 90 Multicast Listener Report Message v2
ARP 60 Who has 172.16.10.141? Tell 172.16.10.156
NBNS 92 Name query NB DESKTOP-M7FO79E<1c>
ARP 60 Who has 172.16.8.47? Tell 172.16.8.73
ARP 60 Who has 172.16.10.222? Tell 172.16.8.73
MDNS 249 Standard query response 0x0000 A, cache flush 172.16.10.25 AAAA, cache flush
MDNS 269 Standard query response 0x0000 A, cache flush 0.0.0.0 A, cache flush 172.16.10.25 AAAA, cache flush
NBNS 92 Name query NB WPAD<00>
BROWSER 243 Host Announcement DESKTOP-E1SFMRV, Workstation, Server, NT Workstation
SSDP 179 M-SEARCH * HTTP/1.1
ARP 60 Who has 172.16.10.144? (ARP Probe)
Frame 9244: Packet, 112 bytes on wire
Ethernet II, Src: WingtechGrou_6d:97:
Internet Protocol Version 6, Src: fe
User Datagram Protocol, Src Port: 53
Multicast Domain Name System (query)
wireshark_EthernetFSRJR3.pcapng
Packets: 23329 · Dropped: 0 (0.0%)
Profile: Default

- HTTP → http
- DHCP → bootp

5. Inspect the filtered packets in the packet details.

6. Save the captured packets for future analysis.

RESULT: Successfully captured 100 network packets applied display filters for TCP, UDP, ARP, DNS, HTTP, IP, ICMP, and DHCP and analyzed packet details using Wireshark.

Ex. NO.6 ERROR DETECTION AND CORRECTION USING HAMMING CODE

6/08/26

AIM: To write a program to implement error detection and correction using the Hanning code.

ALGORITHM:

1. Start the program
2. Read the 4-bit data.
3. Generate parity bits and form the hanning code.
4. Introduce a single-bit error in the code.
5. Check the received code for errors.
6. If an error is found, identify and correct it.
7. Diplay the corrected code and orignal data.
8. Stop the program.

Code:

```
data = list(map(int, input("Enter 4 bits:").split()))

p1 = data[0]^data[1]^data[3]
p2 = data[0]^data[2]^data[3]
p4 = data[1]^data[2]^data[3]
h = [p1, p2, data[0], p4, data[1], data[2], data[3]]
```

Sample Input: 1 0 1 1

3

Sample output:

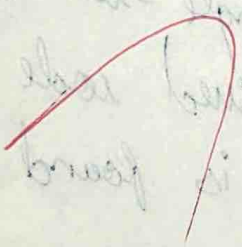
Hamming code: 0 1 1 0 0 1 1

Received code: 0 1 0 0 0 1 1

Error at position: 3

Corrected code: 0 1 1 0 0 1 1

Recovered Data: 1 0 1 1



```
print("Hamming Code:", "".join(map(str, h)))
Pos = int(input("Enter error Pos (1-7), 0 for
                 No error:"))
if Pos:
    h[Pos-1] ^= 1
print("Received Code:", "".join(map(str, h)))
c1 = h[0] ^ h[2] ^ h[4] ^ h[6]
c2 = h[1] ^ h[2] ^ h[5] ^ h[6]
c4 = h[3] ^ h[4] ^ h[5] ^ h[6]
error = c1 + 2*c2 + 4*c4
if error:
    print("Error at Pos:", error)
    h[error-1] ^= 1
    print("corrected code:", "".join(map
                                      (str, h)))
else:
    print("No error")
print("Recovered Data:", h[2], h[4],
                         h[5], h[6])
```

RESULT: The program for error detection and correction using Hamming code was implemented successfully.

AIM: To implement flow control at the data link layer using the sliding window protocol and to simulate the flow of frames from one node to another.

ALGORITHM:

1. Start the program.
2. Read total number of frames, window size and lost frame number.
3. send frames within the current window.
4. If a frame is lost, retransmit from the lost frame.
5. Receive ACK's and slide the window.
6. Repeat until all frames are sent.
7. stop.

code:

```
n = int(input("Enter total frames: "))
w = int(input("Enter window size: "))
lost = int(input("Enter lost frame (-1 for None): "))
i = 0
while i < n:
    print("\n Window:", i, "to", min(i+w-1, n-1))
    for j in range(i, min(i+w, n)):
        print("Sending Frame", j)
```

Sample I/P:

Enter total frames: 8
Enter window size: 4
Enter lost frame (-1 for None): 5

sample o/p:

window : 0 to 3
Sending Frame 0
Sending Frame 1
Sending Frame 2
Sending Frame 3

ACK 0
ACK 1
ACK 2
ACK 3

window : 4 to 7
sending Frame 4
sending Frame 5
Frame 5 Lost
Retransmitting from Frame 5
Sending Frame 5
Sending Frame 6
Sending Frame 7

ACK 4
ACK 5
ACK 6
ACK 7

All i frames transmitted successfully.

```
        if j == lost:
            print("Frame", j, "LOST")
            print("Retransmitting from Frame", j)
            for k in range(j, min(i+w, n)):
                print("sending Frame", k)

        lost = -1
    for j in range(i, min(i+w, n)):
        print("ACK", j)

    i += w
print("All frames transmitted successfully")
```

RESULT: The program to implement Flow control at the Data Link Layer using the sliding Window Protocols was successfully implemented.

nmap -PR -sn TARGET

Case: Host is live

nmap -PE -sn TARGET

Case: Host is live.

nmap -PP -sn TARGET

Case: Host is live.

nmap -PM -sn TARGET

Case: Host is live.

TCP 3-Way Handshake

Case: TCP port is open.

Ex. 8(a)

DISCOVER NVE HOSTS USING NMAP

AIM:

To discover live hosts in a network using Nmap by performing ARP, ICMP, TCP and UDP-based host discovery on the TryHackMe Nmap room.

PROCEDURE:

1. Open the TryHackMe Nmap room and start the AttackBox.
2. Identify the target IP address/subnet to be scanned.
3. Use Nmap host-discovery options to identify active systems.
4. Perform ARP-based discovery using:

   nmap -PR -sn <target>
5. Perform ICMP Echo discovery using the -PE option and study ICMP Timestamp (-PP) and Address Mask (-PM) discovery.
6. Perform TCP SYN and TCP ACK host discovery using -PS and -PA. study UPP-based host discovery using -PU and observe ICMP Port unreachable responses.
7. Use reverse-DNS lookup options to obtain hostnames where applicable.

8. Record the live hosts and verify the responses obtained from the servers.

RESULT: Thus, the live hosts in the network were successfully discovered using Nmap and the responses from the active hosts were recorded.

Ex.P8(a)

CONFIGURATION OF WIRELESS LAN USING CISCO PACKET TRACER

AIM:

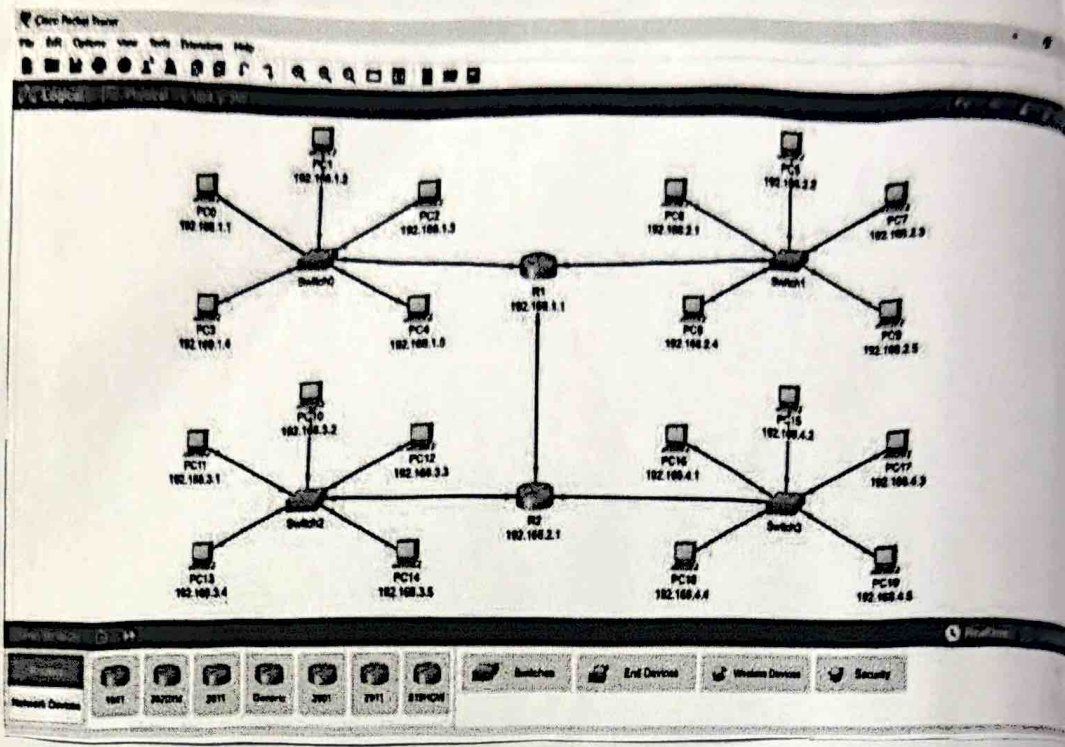
To design and configure a wireless LAN using Cisco Packet Tracer with three PCs connected to a Linksys Wireless Router.

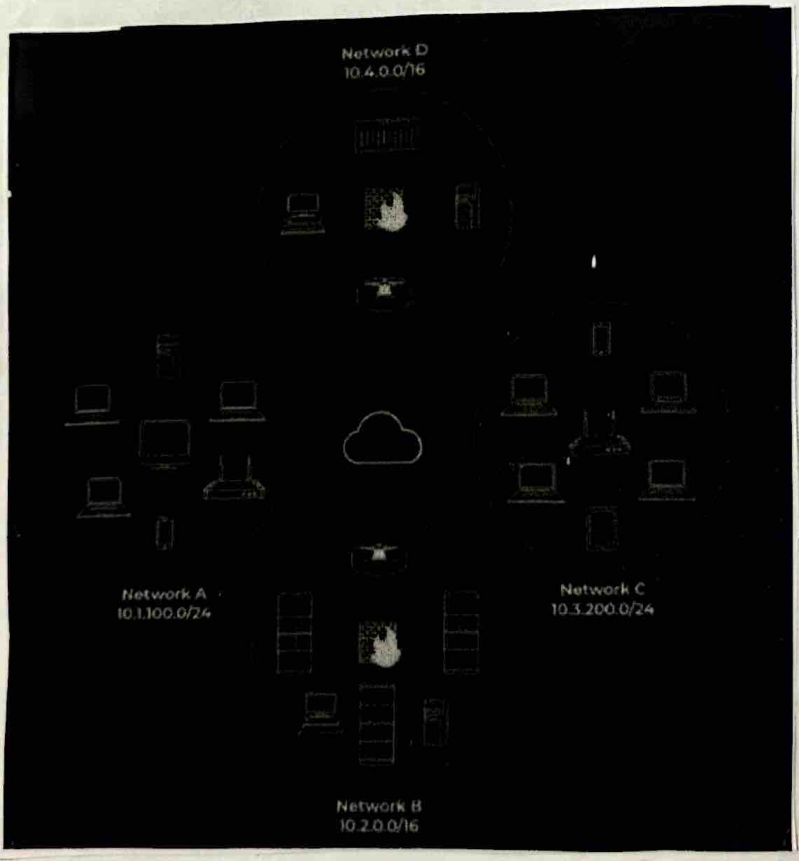
ALGORITHM:

1. Open Cisco Packet Tracer and create a new workspace.
2. Place a Linksys Wireless Router and three PCs.
3. configure the router with the required network settings.
4. set the Wireless SSID as MotherNetwork.
5. Configure WEP security for the wireless Network
6. Open the Wireless settings of the first PC and select MotherNetwork.
7. Enter the security Key and connect the PC to the router.
8. Repeat the same configuration for the other two PCs.
9. Verify that all the 3PCs are connected to wireless network.
10. Test communication between the connected PCs.

RESULT:

Thus, the Wireless LAN was successfully configured using Cisco Packet Tracer with 3 PCs connected to the Linksys Wireless Router.


PC0
PC1
PC2
PC3
PC4
Switch0
R1
PC5
PC6
PC7
PC8
PC9
Switch1
PC10
PC11
PC12
PC13
PC14
Switch2
R2
PC15
PC16
PC17
PC18
PC19
Switch3
Network Devices
Switches
End Devices
Wireless Devices
Security


Network D
10.4.0.0/16
Network A
10.1.100.0/24
Network C
10.3.200.0/24
Network B
10.2.0.0/16

SUBNETTING

2PC, 2 SW
1R.

AIM:

To design and configure a subnetted network using Cisco Packet Tracer.

PC1 IP: 192.168.1.130
Sub: 255.255.255.128
Def: 192.168.1.129

P0 -> IP: 192.168.1.2
Sub: 255.255.255.128
Def: 192.168.1.1

ALGORITHM:

1. Open Cisco Packet Tracer.
2. Add a router, switch and PCs.
3. Connect the devices using suitable cables.
4. Open the router CLI.
5. Configure the router interfaces with IP address and subnet masks.
6. Enable the interfaces using no shutdown.
7. Configure the IP addresses and subnet mask for each PC.
8. Set the appropriate default gateway for each PC.
9. Use the Ping command to test connectivity.
10. Verify the communication between the different subnets.

① cmd:

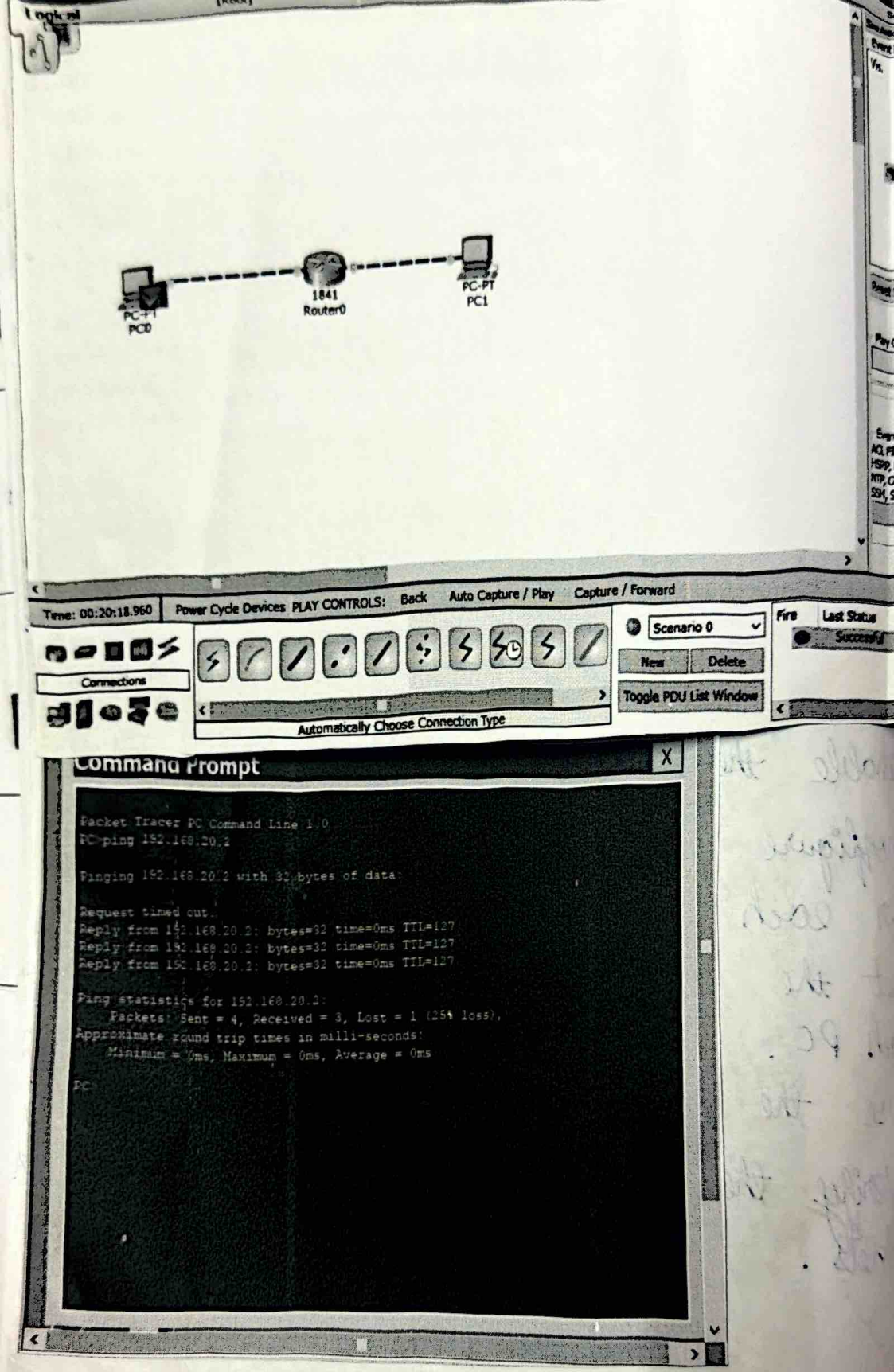
enable
configure terminal
interface gigabitEthernet 0/0
IP address 192.168.1.1 255.255.255.128
no shutdown

② cmd:

interface gigabitEthernet 0/1
IP address 192.168.1.129 255.255.255.128
no shutdown

RESULT:

Thus, the subnetted network was successfully configured and connectivity was verified using Ping.


PC-PT
PC0
1841
Router0
PC-PT
PC1
Time: 00:20:18.960
Power Cycle Devices
PLAY CONTROLS:
Back
Auto Capture / Play
Capture / Forward
Connections
Automatically Choose Connection Type
Scenario 0
New
Delete
Toggle PDU List Window
Fire
Last Status
Command Prompt
Packet Tracer PC Command Line 1.0
PC>ping 192.168.20.2
Pinging 192.168.20.2 with 32 bytes of data:
Request timed out.
Reply from 192.168.20.2: bytes=32 time=0ms TTL=127
Reply from 192.168.20.2: bytes=32 time=0ms TTL=127
Reply from 192.168.20.2: bytes=32 time=0ms TTL=127
Ping statistics for 192.168.20.2:
Packets: Sent = 4, Received = 3, Lost = 1 (25% loss),
Approximate round trip times in milli-seconds:
Minimum = 0ms, Maximum = 0ms, Average = 0ms
PC>

EXP. 10(a)

INTERNETWORKING WITH ROUTER.

AIM:

To design and configure an interaetwor using a router in Cisco Packet Tracer.

ALGORITHM:

2PC & 1R

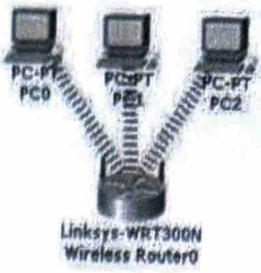
PC0: IP: 192.168.10.2

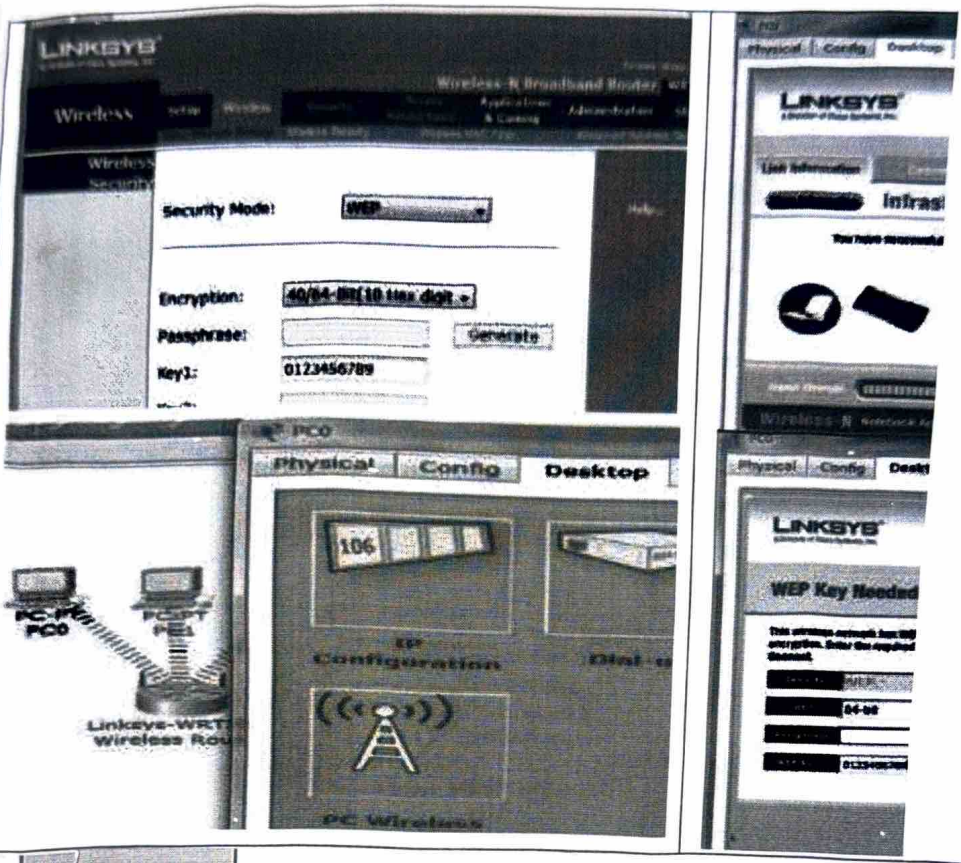
PC1: IP: 192.168.20.2

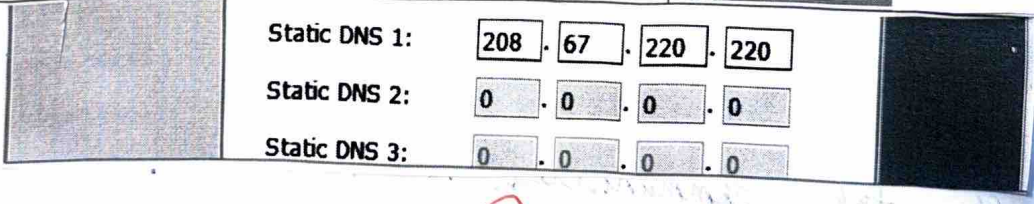
1. Open Cisco Packet Tracer.
2. Add one router and two PCs.
3. Connect the PCs to the router.
4. Configure the router interfaces with IP addresse
5. Enable the interfaces using no shutdown.
6. Configure IP address and subnet mask for PC0 and PC1.
7. set the default gateways for both PCs.
8. Send a PDU from PC0 to PC1.
9. Verify the communication between the two networks.

RESULT:

Thus, the internetwork was successfull configured using a router and connectivity was verified.


PC-PT
PC0
PC-PT
PC1
PC-PT
PC2
Linksys-WRT300N
Wireless Router0


LINKSYS
Wireless
Security Mode:
WEP
Encryption:
Passphrase:
Generate
Key1:
0123456789
PC0
Physical
Config
Desktop
106
EP
Configuration
PC Wireless
LINKSYS
WEP Key Needed


Static DNS 1:
208 . 67 . 220 . 220
Static DNS 2:
0 . 0 . 0 . 0
Static DNS 3:
0 . 0 . 0 . 0

WIRELESS INTERNETWORKING

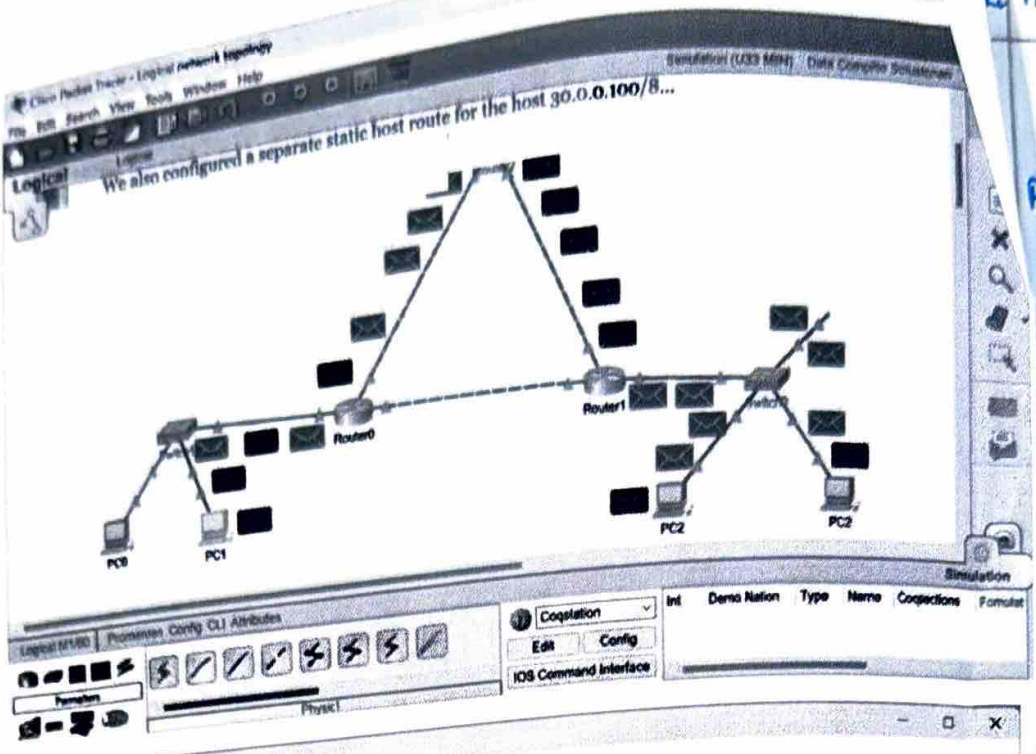
AIM:

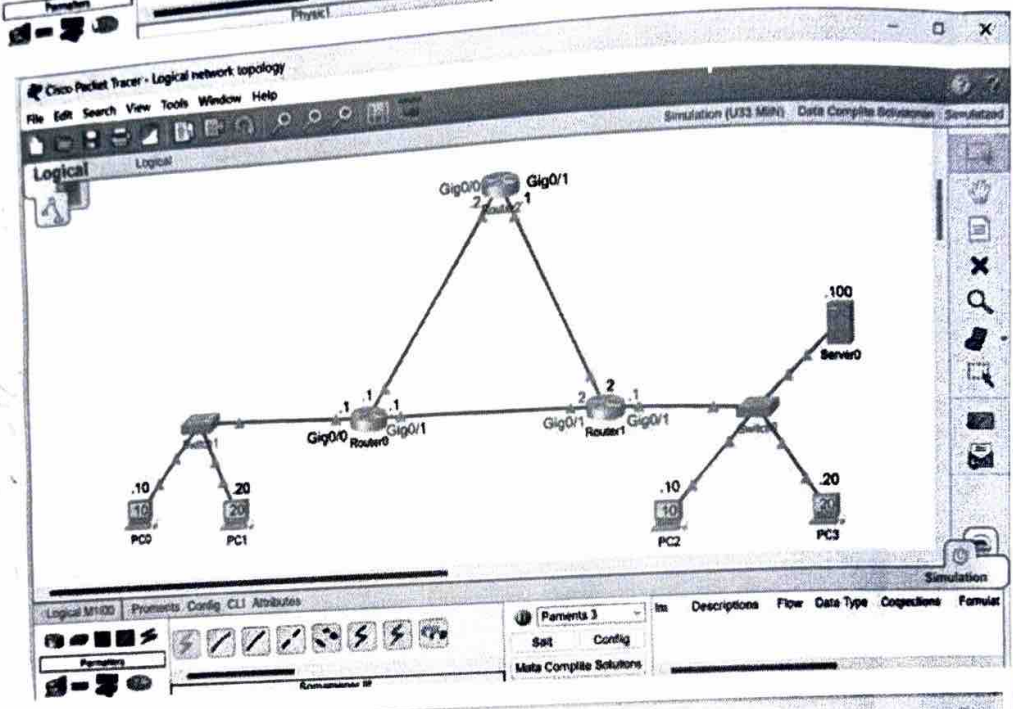
To design and configure an internetwork using a wireless router, DHCP server and Internet cloud.

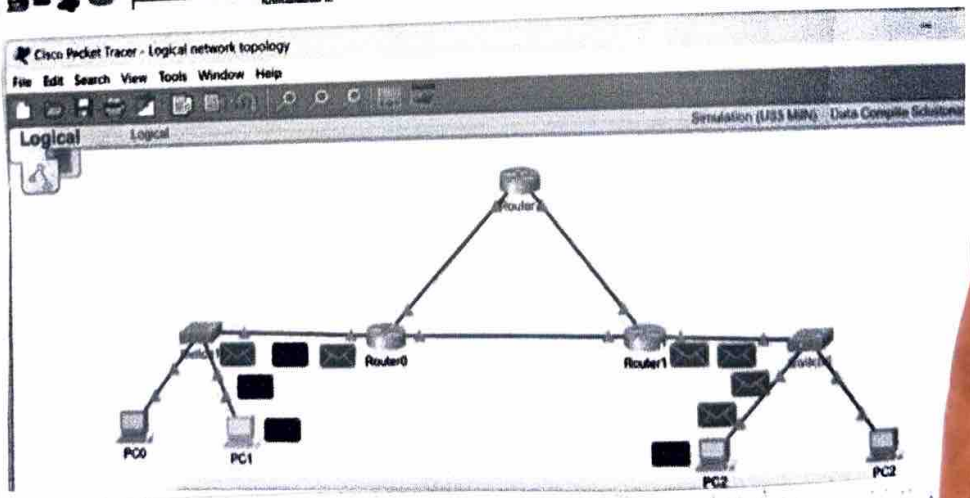
ALGORITHM:

1. Open Cisco Packet Tracer.
2. Add a Wireless router, DHCP server and Internet cloud.
3. Add the required wireless devices.
4. Connect the devices to the wireless router.
5. Configure the wireless router settings.
6. Configure the DHCP server with IP settings.
7. Enable DHCP for automatic IP assignment.
8. Connect the wireless devices to the network.
9. Verify the assigned IP addresses.
10. Test connectivity between the devices and Internet cloud.

RESULT: Thus, the wireless internetwork was successfully configured and connectivity was verified.


We also configured a separate static host route for the host 30.0.0.100/8...
Router0
Router1
PC0
PC1
PC2
PC2


Cisco Packet Tracer - Logical network topology
Logical
Gig0/0
Gig0/1
Router0
Router1
Server0
.100
.10
.20
PC0
PC1
PC2
PC3


Cisco Packet Tracer - Logical network topology
Logical
Router0
Router1
PC0
PC1
PC2
PC2

EX. 11(a)

STATIC ROUTING

AIM: To configure static routing using Cisco Packet Tracer.

ALGORITHM:

1. Open Cisco Packet Tracer.
2. Create the required router and PC topology.
3. Connect the devices using suitable cables.
4. Assign IP addresses to PCs and the router interfaces.
5. Enable router interfaces using no shutdown.
6. Configure static routes using ip router.
7. Configure the required backup route.
8. Check the routing table using show ip route
9. Test connectivity using ping.
10. Verify communication between the networks.

show

RESULT:

Thus, static routing was successfully configured and connectivity was verified.

EX II(a)

RIP ROUTING

AIM:

To configure RIP routing using Cisco Packet Tracer.

ALGORITHM:

1. Open Cisco Packet Tracer.
2. Create the required routers and PC topology.
3. Connect the devices using suitable cables.
4. Assign IP address to PCs and router interfaces.
5. Enable router interfaces using no shutdown.
6. Enter RIP configuration mode using router rip.
7. Add the connected networks using network commands.
8. Configure RIP on the required routers.
9. Check the routing table.
10. Test connectivity using Ping.

RESULT:

Thus, RIP routing was successfully configured and connectivity between the networks was verified.

Client.py

```
import socket
s = socket.socket()
s.connect(('localhost', 1234))
msg = input("Enter message: ")
s.send(msg.encode())
print("Echo: ", s.recv(1024).decode())
s.close()
```

server o/p: client connected

client o/p: Enter message: Hello
Echo: Hello

EX. 12 (a)

ECHO CLIENT - SERVER

AIM:

To implement an echo client - server using TCP / UDP sockets.

ALGORITHM:

1. Create the server socket.
2. Bind the socket to an IP address and port.
3. Start listening for client requests.
4. Create the client socket.
5. Connect the client to the server.
6. Enter a message from the client.
7. Send the message to the server.
8. Server receives and echoes the message.
9. Client receives the echoed message.
10. Close the sockets.

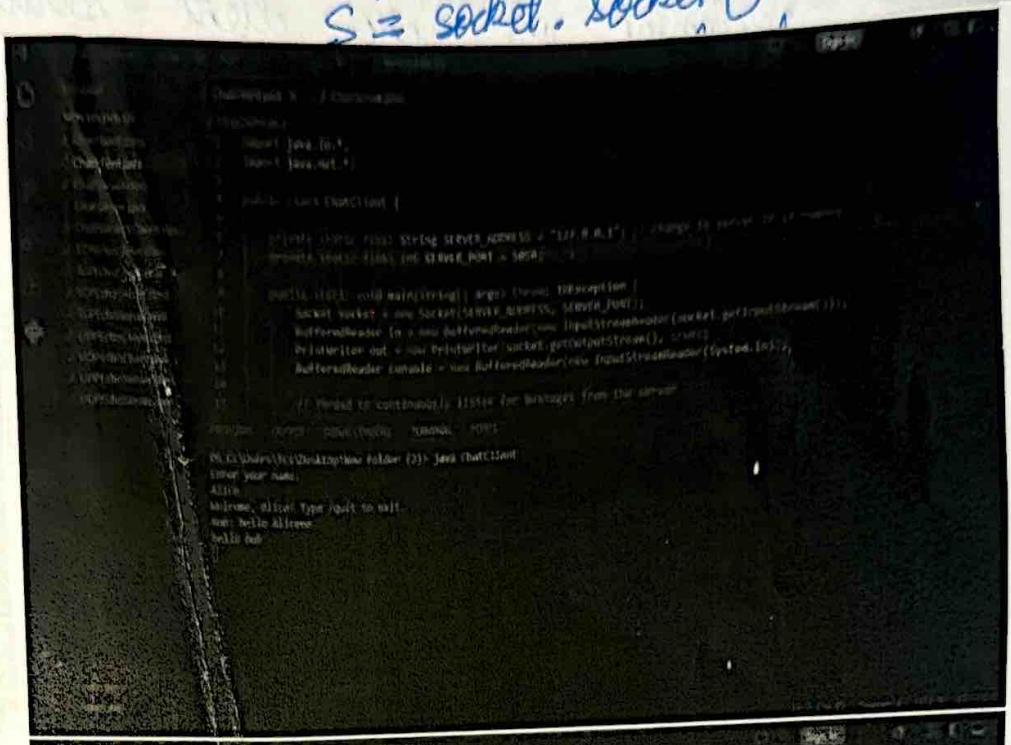
RESULT:

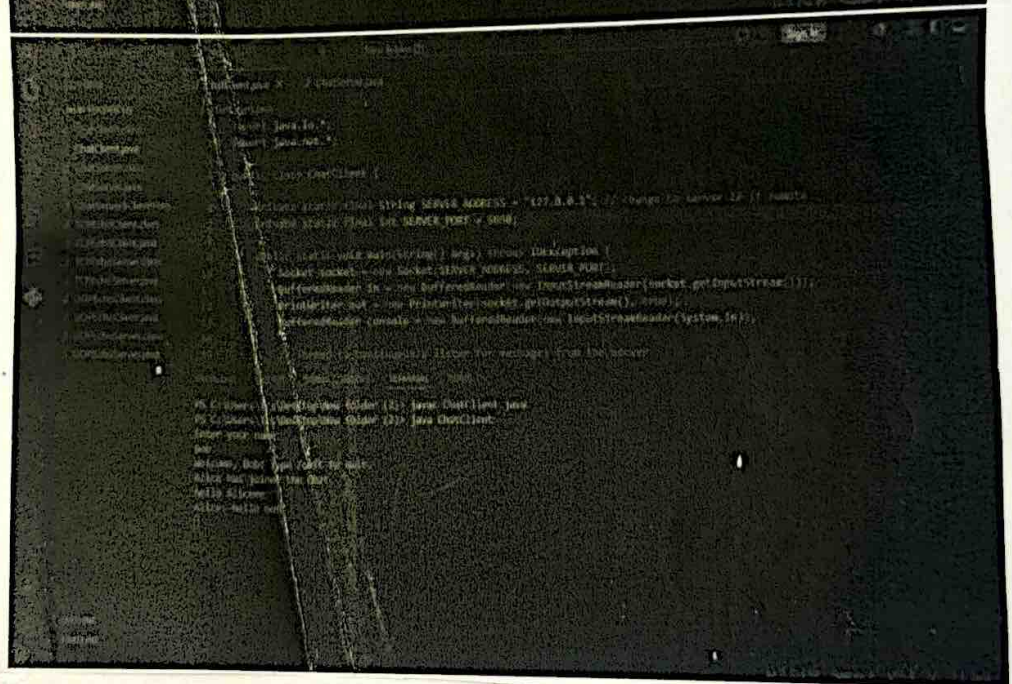
Thus, the echo client - server was successfully implemented using sockets.

server.py

import socket

s = socket.socket()





s.send(msg.encode())

if msg == "exit":

break

server o/p :

client : Hello

Reply : Hi

client : How are you?

Reply : Fine

s.close

client o/p :

Message : Hello

Server : Hi

Message : How are you?

Server : Fine

Message : exit.

Ex.

1.

2.

3.

4.

5.

6.

7.

8.

9.

10.

SERVER

AIM:

To implement a chat program using socket programming.

ALGORITHM:

1. Create the server socket.
2. Bind the socket to an IP address and port.
3. Start listening for a client.
4. Create the client socket.
5. Establish connection between client and server.
6. Enter a message from the client.
7. Send and receive messages between client and server.
8. Display the received messages.
9. Continue communication until termination.
10. Close the sockets.

RESULT:

Thus, the chat client-server was successfully implemented using socket programming

code:

```
import socket
import struct
import time
import os

def ping(host):
    try:
        ip = socket.gethostbyname(host)
        s = socket.socket(socket.AF_INET,
            socket.SOCK_RAW,
socket.IPPROTO_ICMP)

        pid = os.getpid() & 0xffff
        header = struct.pack("!BBHHH", 8, 0, pid, 1)
        data = b"PINGS"
        total = sum((data[i] << 8) + data[i+1]
        for i in range(0, len(data), 2))
            total = (total >> 16) + (total & 0xffff)
            checksum = (~total) & 0xffff
            packet = struct.pack("!BBHHH", 8,0,
        checksum, pid, 1) + data
            start = time.time()
            s.settimeout(2)
            s.sendto(packet, (ip, 1))
            s.recvfrom(1024)
            print("Reply from", ip, "time:",
round((time.time() - start)*1000, 2), "ms"))
```

AIM: To implement a simple Ping Program using socket Programming to check whether a destination host is reachable and to calculate the response time.

ALGORITHM:

* Start the program.
* Get destination IP address from User.
* Create a raw socket using ICMP Protocol.
* Construct an ICMP Echo Request Packet.
* Record the time before sending Packet.
* Send the ICMP Packet to the destination host.
* Wait for the ICMP Echo reply.
* Record the time when reply is received.
* Calculate the Round Trip Time (RTT).
* Display whether the host is reachable, show response time.
* Close the socket and terminate the Program.

```
except socket.timeout:
    print ("Request timed out")
except PermissionError:
    print ("permission denied")

Ping (input ("Enter Destination host: "))
```

I/P:

Enter IP address: 8.8.8.8

O/P:

Host name: dns.google.
IP address: 8.8.8.8

RESULT :: This a ping Program was implemented successfully using socket Programming to determine host reachability and calculate round trip time.

code:

```
import socket
import struct

s = socket.socket(socket.AF_INET,
    socket.SOCK_RAW, socket.IPPROTO_IP)
s.bind(("0.0.0.0",0))
s.setsockopt(socket.IPPROTO_IP,
    socket.IP_HDRINCL,1)
print("Packet Sniffer started...")
while True:
    packet, addr = s.recvfrom(65535)
    ip = struct.unpack("!BBHHHBBH4s4s",
     packet[:20])
print("Source IP:", socket.inet_ntoa(ip[8]))
print("Destination IP:", socket.inet_ntoa
                          (ip[9]))
print("Protocol:", ip[6])
print("Packet size:", len(packet), "bytes")
```

O/p:

```
Packet Sniffer started.....
Source IP: 192.168.1.10
Destination IP: 142.250.183.14
Protocol: 6
Packet Size: 60 Bytes
```

Ex.No.14 PACKET SNIFFING USING RAW SOCKETS

AIM: To implement a Packet sniffing Program using raw socket and capture network Packets to analyze their basic information such as source address, distination and Protocol.

ALGORITHM:

* Start the Program.
* Create rawSocket for capturing network Packets.
* Bind the socket to the required network interface.
* Extract the IP header from each Packet.
* Identify the source IP address.
* Determine the Protocol type such as TCP, UDP, ICMP.
* Display the Packet information.
* Continue capturing Packet until the user stops the Program.
* close the raw socket.
* Terminate the Program.

RESULT: Thus, the Ping Program and Packet sniffing Program were successfully implemented using sockets.

I/P :

Client IP, Date & Time, URL, HTTP Method, HTTP Status Code, Bytes Transferred, Referrer, User Agent, access.log

O/P :

Webalizer statistics

Total Requests : 1250
Total Visits : 480
Total Bytes : 15.4 MB
Successful Requests : 1180
404 Errors : 45

EX.15

ANALYSE VARIOUS TYPES OF SERVICES USING WEBALIZER TOOL.

AIM: TO analyse web server log files using the webalizer tool and study various server statistics.

ALGORITHM:

* Start the Webalizer application.
* Install and configure Webalizer on the system.
* Locate the Web server access log file.
* Set the log file path and output directory.
* Run Webalizer on the log file.
* Generate statistics like visits, request, pages accessed, and data transferred.
* Open the generated HTML report.
* Analyze graphs and tables.
* Compare server statistics.
* Record observations.

RESULT: Webalizer was successfully used to analyse web server log files and generate server statistics.